



CS-317



Operating Systems

LECTURE 13



Concurrency

BOOK 1 CHAPTER 5

In the last lecture...



- ▶ We looked in detail at all the ways processes interact.
- ▶ We discussed examples of deadlock, starvation and data coherence.
- ▶ We realized the requirements of ME.
- ▶ We delved into the hardware approach to implement ME.



Mutual Exclusion

HARDWARE SUPPORT

How to satisfy the requirements of ME?

- ▶ Leave the responsibility with the processes that wish to execute concurrently.
 - ▶ We can refer to this as the **software approach**.
 - ▶ This approach is prone to high processing overhead and bugs.
- ▶ Make use of special purpose machine instructions.
 - ▶ We can refer to this as the **hardware approach**.
 - ▶ They reduce overheads.
 - ▶ They are unattractive as a general-purpose solution.
- ▶ Provide some level of support within the OS or a programming language.

Special machine instructions

- ▶ There are two machine instructions which deserve a mention here:
 - ▶ Compare and swap 
 - ▶ Exchange

Compare & swap instruction

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval ;
    oldval = *word;
    if (oldval == testval) *word = newval;
    return oldval;
}
```

ATOMIC



- ▶ It is also called compare and exchange instruction.
- ▶ It checks a memory location `*word` against a test value `testval`.
- ▶ If the memory location's current value is equal to `testval`, it is replaced with `newval`; otherwise it is left unchanged.
- ▶ The old memory value is always returned.
- ▶ If the returned value is equal to `testval`, the memory location has been updated.

```
/* program mutual exclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}

void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```



ME using compare & swap

- ▶ **bolt** is a global variable.
- ▶ In the **main** function, before the creation of processes, **bolt** is initialized to **0**.
- ▶ The **parbegin** construct creates **n** processes that will execute code **P**, and admits them to the ready queue, all at the same time.


```
/* program mutual exclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */
        bolt = 0;
        /* remainder */
    }
}

void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

ME using compare & swap

- ▶ A process trying to get into critical section fetches the value of **bolt** and compares it with **0**.
- ▶ If the comparison is successful, **bolt** is updated to **1**, value returned is **0**, so the while condition becomes false, and the process enters critical section.

```
/* program mutual exclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */
        bolt = 0;
        /* remainder */
    }
}

void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

ME using compare & swap

- ▶ If **bolt** was previously 1, the compare returns false, no swap occurs, value returned is 1, and the executing process is caught in the while loop, in which it keeps executing compare and swap repeatedly.
- ▶ When a process leaves critical section, it resets **bolt** back to 0.

Example run of the ME code using compare and swap

- ▶ Consider three processes **P1**, **P2** and **P3**, trying to enter their critical sections and using the compare and swap instruction to implement ME.
- ▶ If the code is correct, ME will be enforced, no matter what assumptions we make regarding the relative speeds of these processes.
- ▶ Points to remember:
 - ▶ Compare and swap compares **bolt** with **0**. If the comparison is true, it swaps **bolt** with **1**.
 - ▶ Compare and swap always returns the old value of **bolt**.
- ▶ For the purpose of demonstration, we assume **P3** to be the fastest.




```
/*program mutual exclusion*/  
const int n = /*number of processes*/;  
int bolt;  
void P(int i) {  
    while (true) {  
        while (compare_and_swap(bolt, 0, 1) == 1)  
            /*do nothing*/;  
        /*critical section*/;  
        bolt = 0;  
        /*remainder*/;  
    }  
}  
void main() {  
    bolt = 0;  
    parbegin (P(1), P(2), ... ,P(n));  
}
```



```
/*program mutual exclusion*/
```

```
const int n = /*number of processes*/;
```

```
int bolt;
```

```
void P(int i) {
```

```
    while (true) {
```

```
        while (compare_and_swap(bolt, 0, 1) == 1)
```

```
            /*do nothing*/;
```

```
        /*critical section*/;
```

```
        bolt = 0;
```

```
        /*remainder*/;
```

```
    }
```

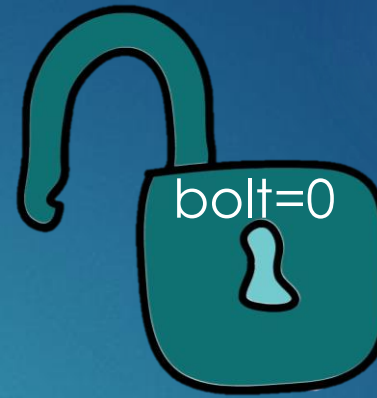
```
}
```

```
void main() {
```

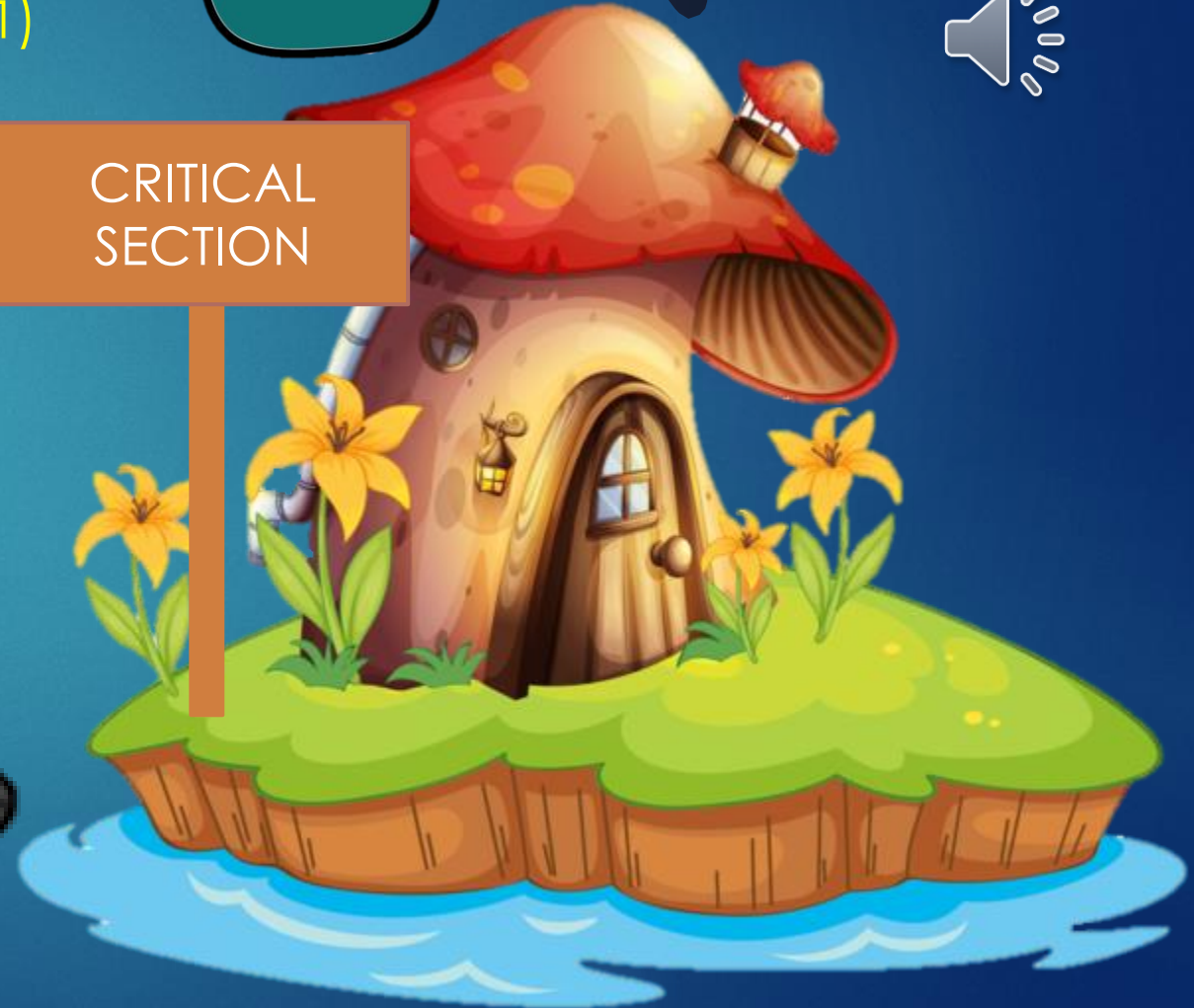
```
    bolt = 0;
```

```
    parbegin (P(1), P(2), ... ,P(n));
```

```
}
```



CRITICAL
SECTION

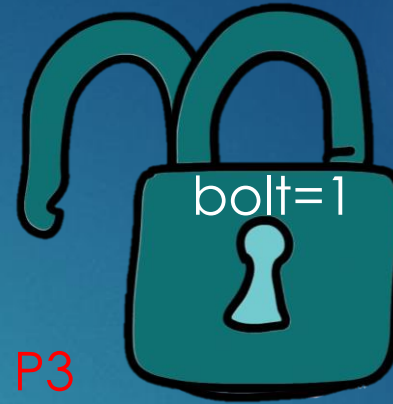



```

/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/;
        bolt = 0;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}

```

0



P3

CRITICAL
SECTION



p3 was the fastest hence came first


```

/*program mutual exclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/;
        bolt = 0;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}

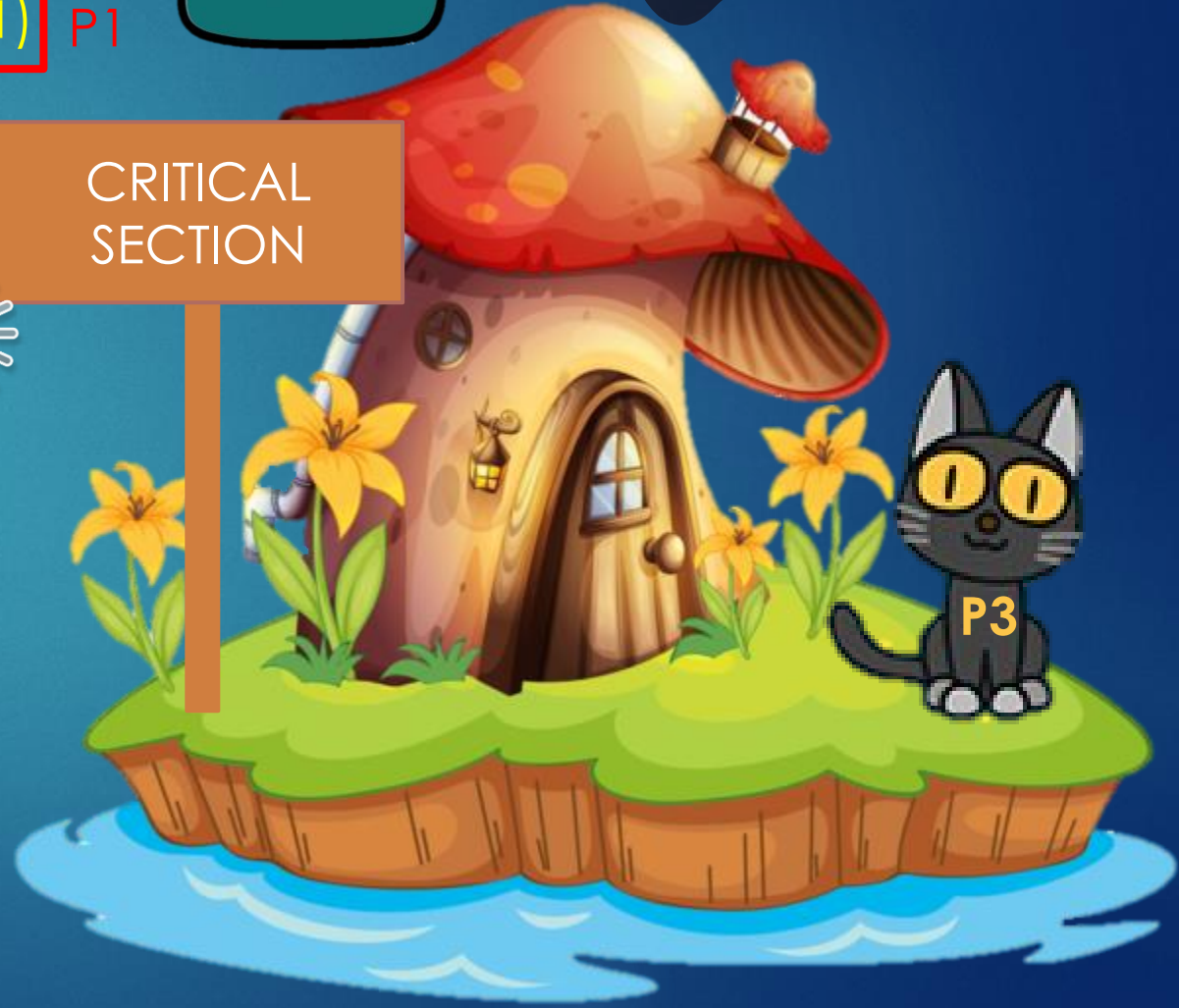
```



P1



CRITICAL
SECTION



```
/*program mutualexclusion*/  
const int n = /*number of processes*/;  
int bolt;
```

```
void P(int i) {  
    while (true) {  
        while (compare_and_swap(bolt, 0, 1) == 1)  
            /*do nothing*/;  
        /*critical section*/;  
        bolt = 0;  
        /*remainder*/;  
    }  
}
```

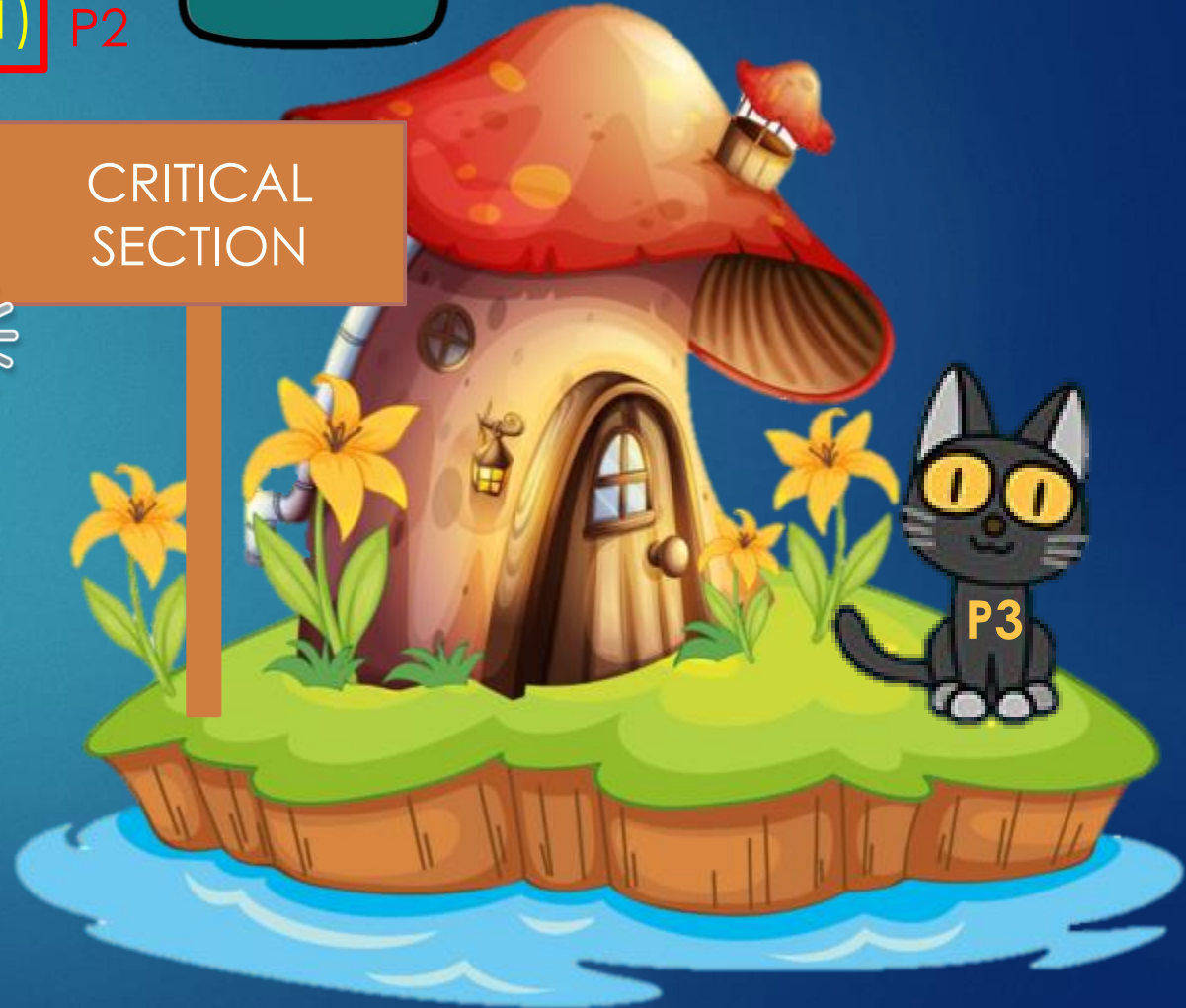
```
void main() {  
    bolt = 0;  
    parbegin (P(1), P(2), ..., P(n));  
}
```



P2



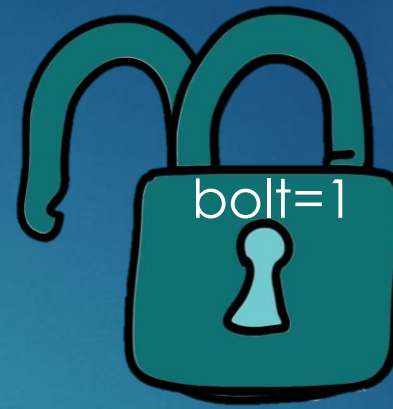
CRITICAL
SECTION




```

/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/; P3
        bolt = 0; P3
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}

```



CRITICAL
SECTION

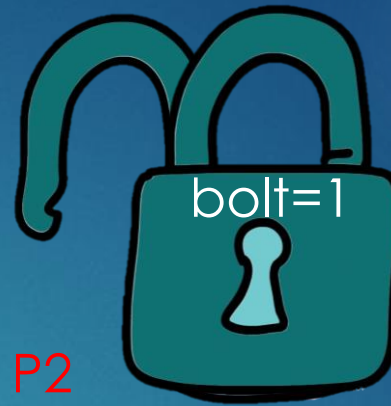



```
/*program mutualexclusion*/  
const int n = /*number of processes*/;  
int bolt;
```

```
void P(int i) {  
    while (true) {  
        while (compare_and_swap(bolt, 0, 1) == 1)  
            /*do nothing*/;  
        /*critical section*/;  
        bolt = 0;  
        /*remainder*/;  
    }  
}
```

```
void main() {  
    bolt = 0;  
    parbegin (P(1), P(2), ..., P(n));  
}
```

0



P2

CRITICAL
SECTION



```
/*program mutualexclusion*/  
const int n = /*number of processes*/;  
int bolt;
```

```
void P(int i) {
```

```
    while (true) {
```

```
        while (compare_and_swap(bolt, 0, 1) == 1)
```

```
            /*do nothing*/;
```

```
            /*critical section*/;
```

```
            bolt = 0;
```

```
            /*remainder*/;
```

```
    }
```

```
}
```

```
void main() {
```

```
    bolt = 0;
```

```
    parbegin (P(1), P(2), ..., P(n));
```

```
}
```



CRITICAL
SECTION




```

/*program mutual exclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/; P2
        bolt = 0; P2
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}

```



CRITICAL
SECTION

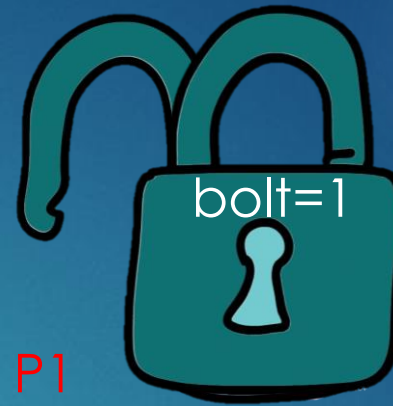



```

/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/;
        bolt = 0;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}

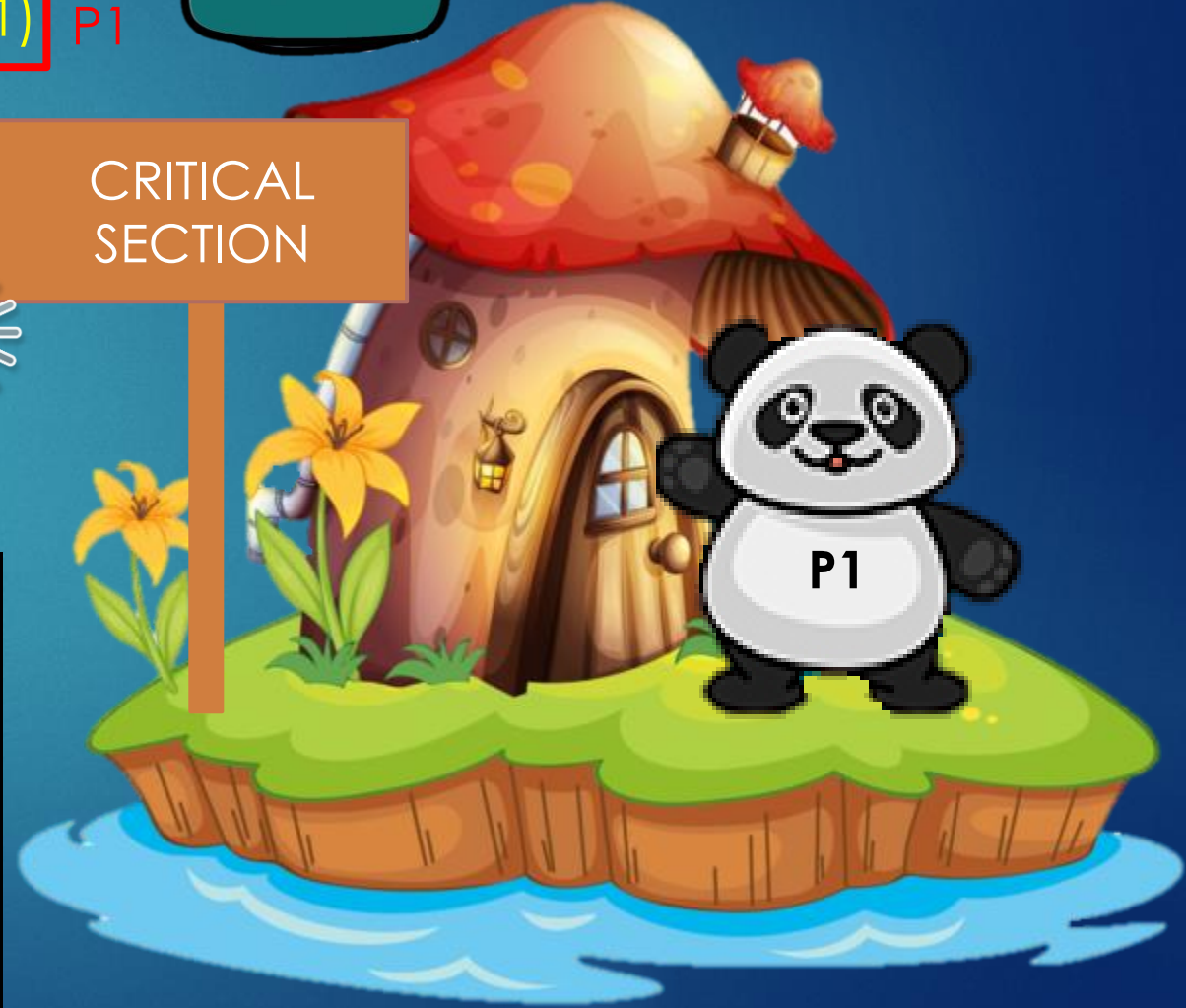
```

0



P1

CRITICAL
SECTION



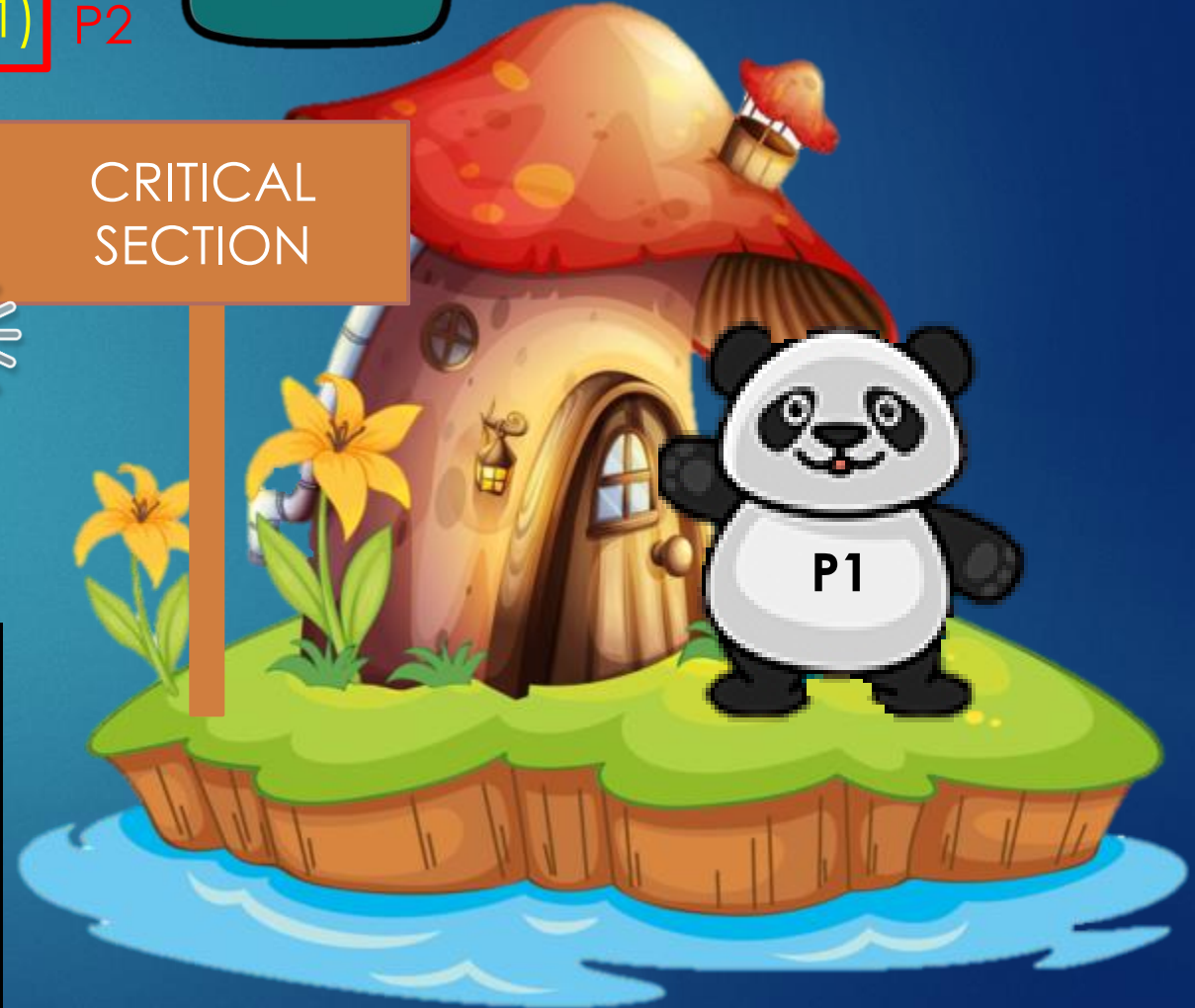
```

/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/;
        bolt = 0;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}

```



CRITICAL
SECTION





Parallel trace of the compare and swap example

```
void P(int i) {  
  while (true) {  
    while (compare_and_swap(bolt, 0, 1) == 1)  
      /*do nothing*/;  
    /*critical section*/;  
    bolt = 0;  
    /*remainder*/; } }
```

the faster process gets the notification first on changing the value of bolt, that is why the p3 and p2 entr critical earlier as comapred to p1

bolt	P1	P2	P3
0			
1			compare & swap
	compare & swap		critical section
		compare & swap	
0			leaves
1		compare & swap	
		critical section	compare & swap
0		leaves	
1	compare & swap		
	critical section	compare & swap	

Busy waiting

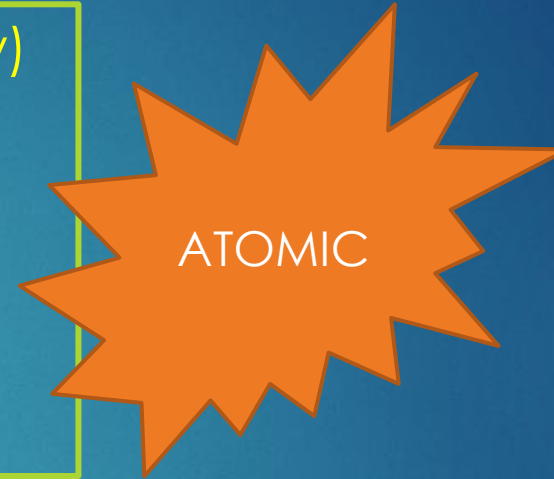


The term **busy waiting**, or **spin waiting**, refers to a technique in which a process can do nothing until it gets permission to enter its critical section but continues to execute an instruction or set of instructions that tests the appropriate variable to gain entrance.

Exchange instruction

compare and swap are atomic instructions too

```
void exchange (int *register, int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```



- ▶ The instruction exchanges the contents of a register with that of a memory location.
- ▶ Both the Intel IA-32 architecture (Pentium) and the IA-64 architecture (Itanium) contain an XCHG instruction.

```
/*program mutualexclusion*/  
const int n = /*number of processes*/;  
int bolt;
```

```
void P(int i) {  
    int keyi = 1;  
    while (true) {  
        do exchange (&keyi, &bolt)  
        while (keyi != 0);  
        /*critical section*/;  
        bolt = 0;  
        keyi = 1;  
        /*remainder*/;  
    }  
}
```

```
void main() {  
    bolt = 0;  
    parbegin (P(1), P(2), ... ,P(n));  
}
```

ME using exchange

- ▶ **bolt** is a global variable.
- ▶ In the **main** function, before the creation of processes, **bolt** is initialized to 0.
- ▶ The **parbegin** construct creates **n** processes that will execute code **P**, and admits them to the ready queue, all at the same time.


```
/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    int keyi = 1;
    while (true) {
        do exchange (&keyi, &bolt)
        while (keyi != 0); busy waiting
        /*critical section*/;
        bolt = 0;
        keyi = 1;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}
```

ME using exchange

- ▶ Each process has a local variable **keyi**, which is initialized to 1.
- ▶ A process trying to get into critical section exchanges the value of **bolt** with **keyi**.
- ▶ If **keyi** is equal to 0, the process moves into its critical section.

since bolt = 0 tells that no lock, no process in critical section

```
/*program mutualexclusion*/
const int n = /*number of processes*/;
int bolt;
void P(int i) {
    int keyi = 1;
    while (true) {
        do exchange (&keyi, &bolt)
        while (keyi != 0);
        /*critical section*/;
        bolt = 0;
        keyi = 1;
        /*remainder*/;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}
```

ME using exchange

- ▶ If **keyi** is not equal to 0, the process exchanges **keyi** and **bolt** and checks **keyi**, repeatedly. The process busy waits.
- ▶ When a process leaves critical section, it sets **bolt** back to 0, and **keyi** to 1.
- ▶ The process which makes the exchange right after the departing process proceeds to critical section.

```

void P(int i) {
int keyi = 1;
while (true) {
do exchange (&keyi, &bolt)
while (keyi != 0);
/*critical section*/;
bolt = 0;
keyi = 1;
/*remainder*/; } }

```



Parallel trace of the exchange example

bolt	P1		P2		P3	
	keyi	action	keyi	action	keyi	action
0	1		1		1	
1	0	exchange				
1		crit_sec	1	exchange		
1					1	exchange
0	1	leaves				
1					0	exchange
1	1	exchange				crit_sec
0					1	leaves
1			0	exchange		
1				crit_sec	1	exchange
1						

Advantages of machine-instruction approach

- ▶ It is applicable to **any number of processes** on either a single processor or multiple processors sharing main memory.
- ▶ It is **simple** and therefore **easy to verify**.
- ▶ It can be used to **support multiple critical sections**; each critical section can be defined by its own variable.

Disadvantages of machine-instruction approach

- ▶ **Busy waiting** is employed. While a process is waiting for access to a critical section, it continues to **consume processor time**.
- ▶ **Starvation is possible**. When a process leaves critical section and more than one processes are waiting, the **selection of a waiting process is arbitrary**. Thus, some process could indefinitely be denied access.

Disadvantages of machine-instruction approach

Only resources that the process has explicitly acquired using locks/mutexes/semaphores remain locked when interrupted

- ▶ Deadlock is possible. since, interrupts are enabled
 - ▶ Process **P1** executes the special instruction (e.g., compare & swap, exchange) and enters its critical section.
 - ▶ **P1** is then interrupted to give the processor to **P2**, which has higher priority. 
 - ▶ If **P2** now attempts to use the same resource as **P1**, it will be denied access because of the mutual exclusion mechanism. Thus, it will go into a busy waiting loop.
 - ▶ However, **P1** will never be resumed because it is of lower priority than another ready process, **P2**.
 - ▶ **P1** waits for **P2** to run and finish off.
 - ▶ **P2** waits for **P1** to leave critical section.



Homework:

Draw up parallel execution traces using the following conditions:

1. Use compare and swap with 4 processes, assuming the order of entry into critical section to be: P3, P1, P2, P4.
2. Use exchange with 2 processes, assuming the order of entry into critical section to be: P2, P1, P2, P1.

OS and programming language mechanisms for concurrency

- ▶ There are three mechanisms that are used to provide the support for concurrency and are implemented either the operating system or the programming language:

- ▶ Semaphores,
- ▶ Monitors, and
- ▶ Messages.





Semaphores

CONCEPT, USE AND IMPLEMENTATION

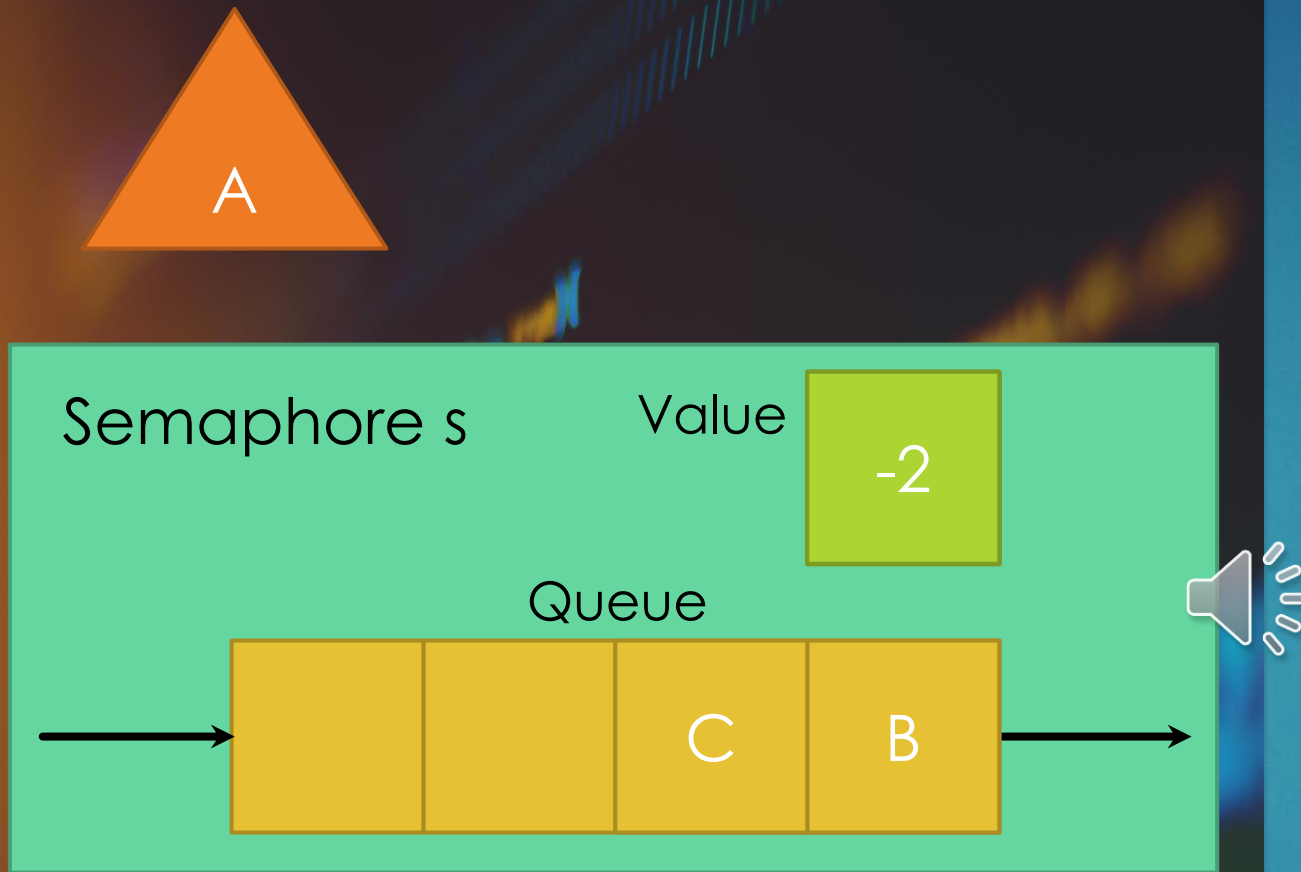
The underlying principle

- ▶ Two or more processes can cooperate by means of simple signals, such that a process can be forced to stop at a specified place until it has received a specific signal.
- ▶ Any complex coordination requirement can be satisfied by the appropriate structure of signals.



Transmission and reception of signals

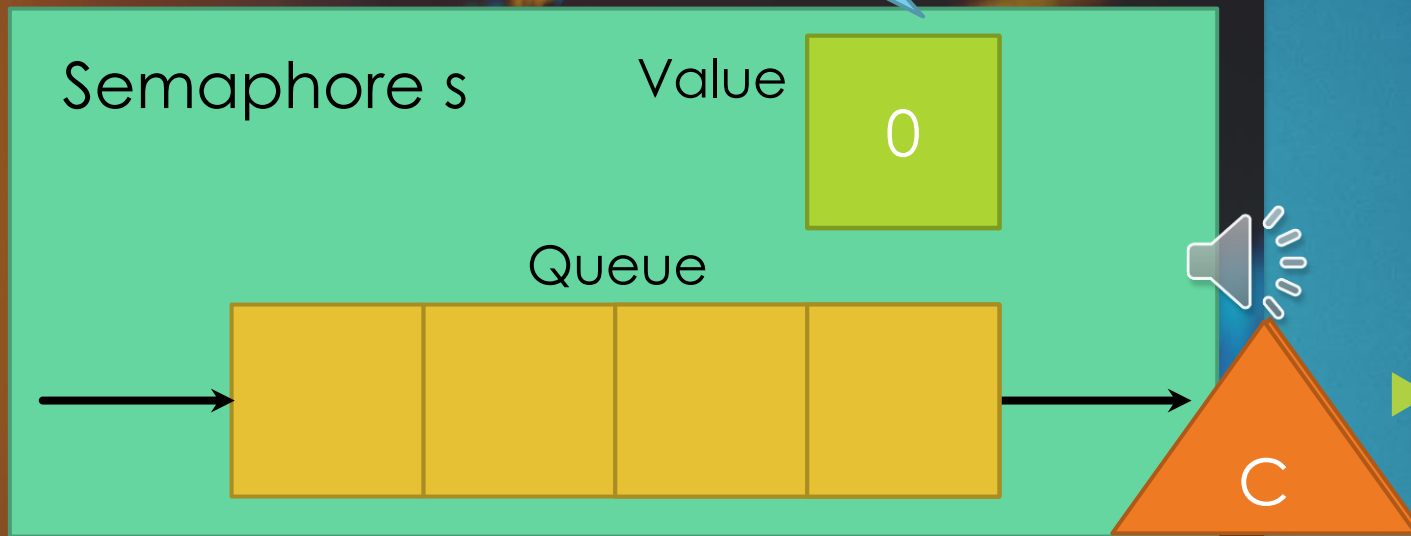
- ▶ To transmit a signal via semaphore s , a process executes the primitive `semSignal(s)`.
- ▶ To receive a signal via semaphore s , a process executes the primitive `semWait(s)`. If the corresponding signal has not yet been transmitted, the process is suspended until the transmission takes place.



Semaphore

- ▶ The semaphore is a **variable** that has an **integer value** upon which only three operations are defined:
 1. A semaphore may be **initialized** to a nonnegative integer value.
 2. The **semWait** operation **decrements the semaphore value**. **If the value becomes negative**, then the **process** executing the **semWait** is **blocked**. Otherwise, the process continues execution.

Semaphore



3. The **semSignal** operation **increments** the semaphore value. If the **resulting value is less than or equal to 0**, then a process blocked by a **semWait** operation, if any, is **unblocked**.

► Other than these three operations (**initialization**, **semSignal** and **semWait**) there is no way to inspect or manipulate semaphores.

```
struct semaphore {  
    int count;  
    queueType queue;  
};  
  
void semWait(semaphore s)  
{  
    s.count--;  
    if (s.count < 0) {  
        /* place this process in s.queue */;  
        /* block this process */;  
    }  
}  
  
void semSignal(semaphore s)  
{  
    s.count++;  
    if (s.count <= 0) {  
        /* remove a process P from s.queue */;  
        /* place process P on ready list */;  
    }  
}
```



Figure 5.3 A Definition of Semaphore Primitives

P and V operations

- ▶ The first major advance in dealing with the problems of concurrent processes came in 1965 with Dijkstra's treatise, in which he proposed the concept of semaphores.
- ▶ In Dijkstra's original paper and in much of the literature,
 - ▶ The letter **P** is used for **semWait**, stands for *proberen* in Dutch, **which means test**.
 - ▶ The letter **V** is used for **semSignal**, stands for *verhogen* in Dutch, **which means increment**.

Significance of the value of a semaphore

- ▶ To begin, the semaphore has a zero or positive value.
- ▶ If the value is positive, that value equals the number of processes that can issue a wait and immediately continue to execute.
- ▶ If the value is zero, either by initialization or because a number of processes equal to the initial semaphore value have issued a wait, the next process to issue a wait is blocked, and the semaphore value goes negative.
- ▶ Each subsequent wait drives the semaphore value further into minus territory.
- ▶ The negative value equals the number of processes waiting to be unblocked.
- ▶ Each signal unblocks one of the waiting processes when the semaphore value is negative.

semaphore positive tells the number of process that can enter critical section without waiting, semaphore zero = no instances of resources available

semaphore value = number of free instances for a resources

Consequences of semaphore definition

- ▶ In general, there is no way to know before a process decrements a semaphore whether it will block or not.
- ▶ After a process increments a semaphore and another process gets woken up, both processes continue running concurrently. There is no way to know which process, if either, will continue immediately on a uniprocessor system.
- ▶ When you signal a semaphore, you don't necessarily know whether another process is waiting, so the number of unblocked processes may be zero or one.

Lessons learned



- ▶ We looked in detail at the compare and swap instruction.
- ▶ We learned how to trace the execution of parallel processes.
- ▶ We looked in detail at the exchange instruction.
- ▶ We conducted an analysis of the machine-instruction approach to mutual exclusion.
- ▶ We learned about semaphores.

Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.